

Análisis y diseño de aplicaciones I

UT3 Diseño y UML



Agenda – Parte 1



Diseño

- Clasificación de sistemas
- Proceso de diseño del software
- Principios del diseño

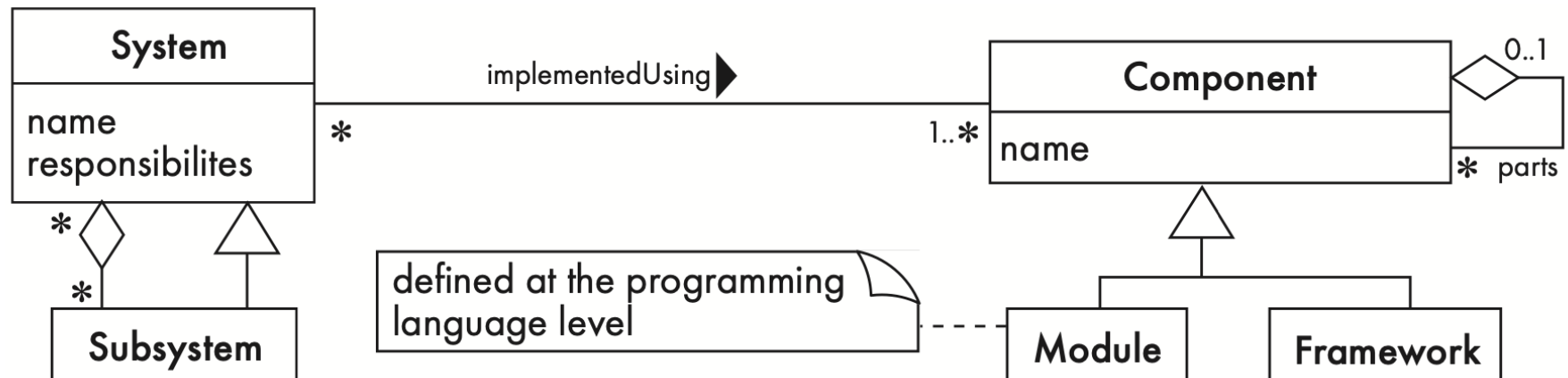
- El **diseño** es tanto el **proceso de** definir:
 - Arquitectura
 - Componentes
 - Interfaces
 - Otras características del sistema o componente
- Y el **resultado** de ese proceso

Definiciones



- Sistema
 - Una **entidad lógica**, que tiene un conjunto de responsabilidades u **objetivos** y consiste de hardware, software o ambos
- Subsistema
 - Un sistema que es **parte de un sistema mayor**,
 - Tiene una interfaz bien definida
- Componente
 - **Cualquier pieza de software o hardware** que tenga definido un rol claro
 - Un componente puede ser aislado, permitiendo que se lo reemplace con un componente diferente que tenga una funcionalidad equivalente
 - Muchos componentes están diseñados para ser **reutilizables**
 - Recíprocamente, otros realizan funciones con un propósito especial
- Módulo
 - Un componente que es definido a nivel de lenguaje de programación.
 - **Package / Namespace**

Definiciones

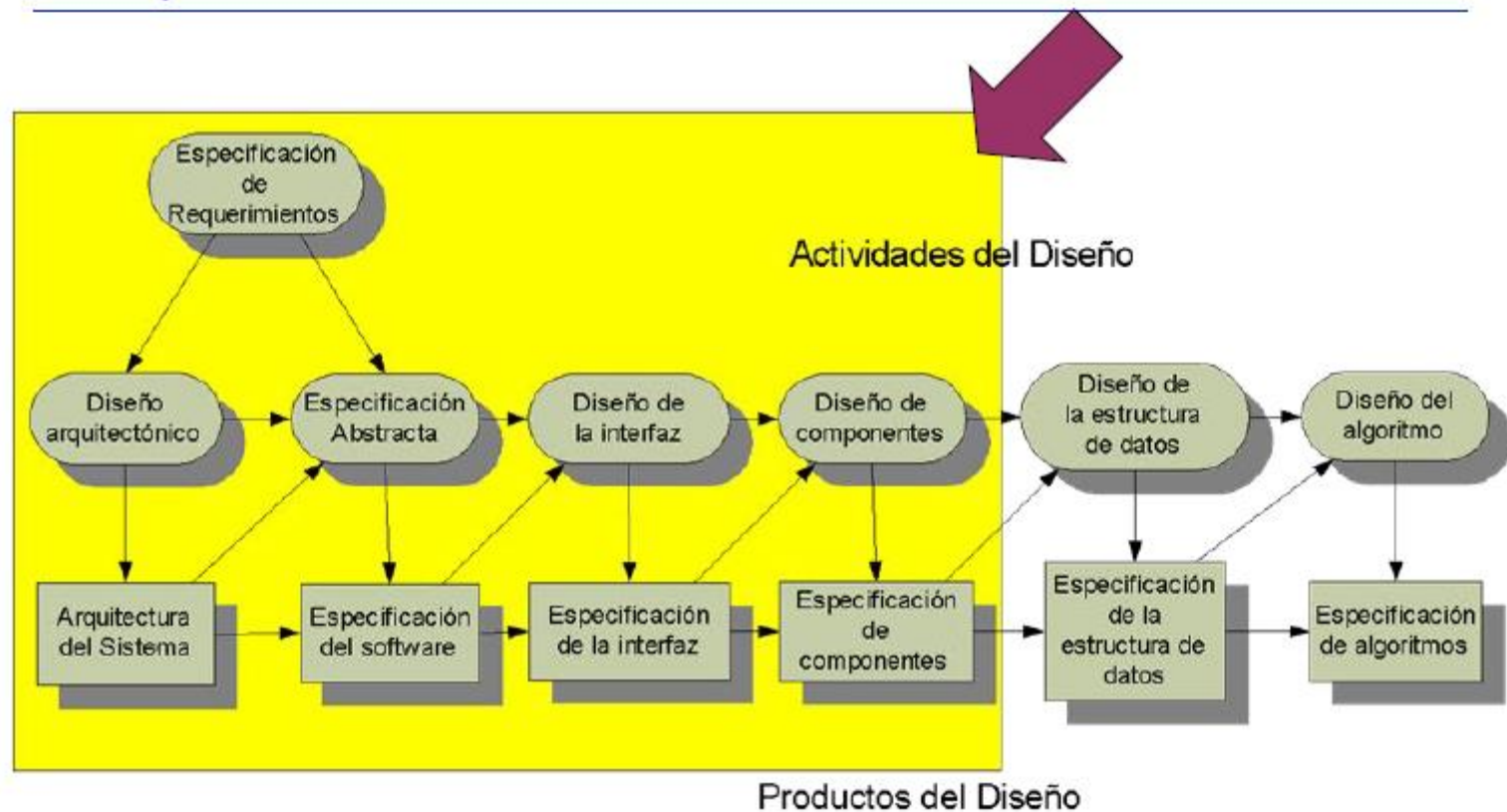


Tarea de Aplicación 1

Clasificación inicial de sistemas.



Modelo del proceso de diseño del software

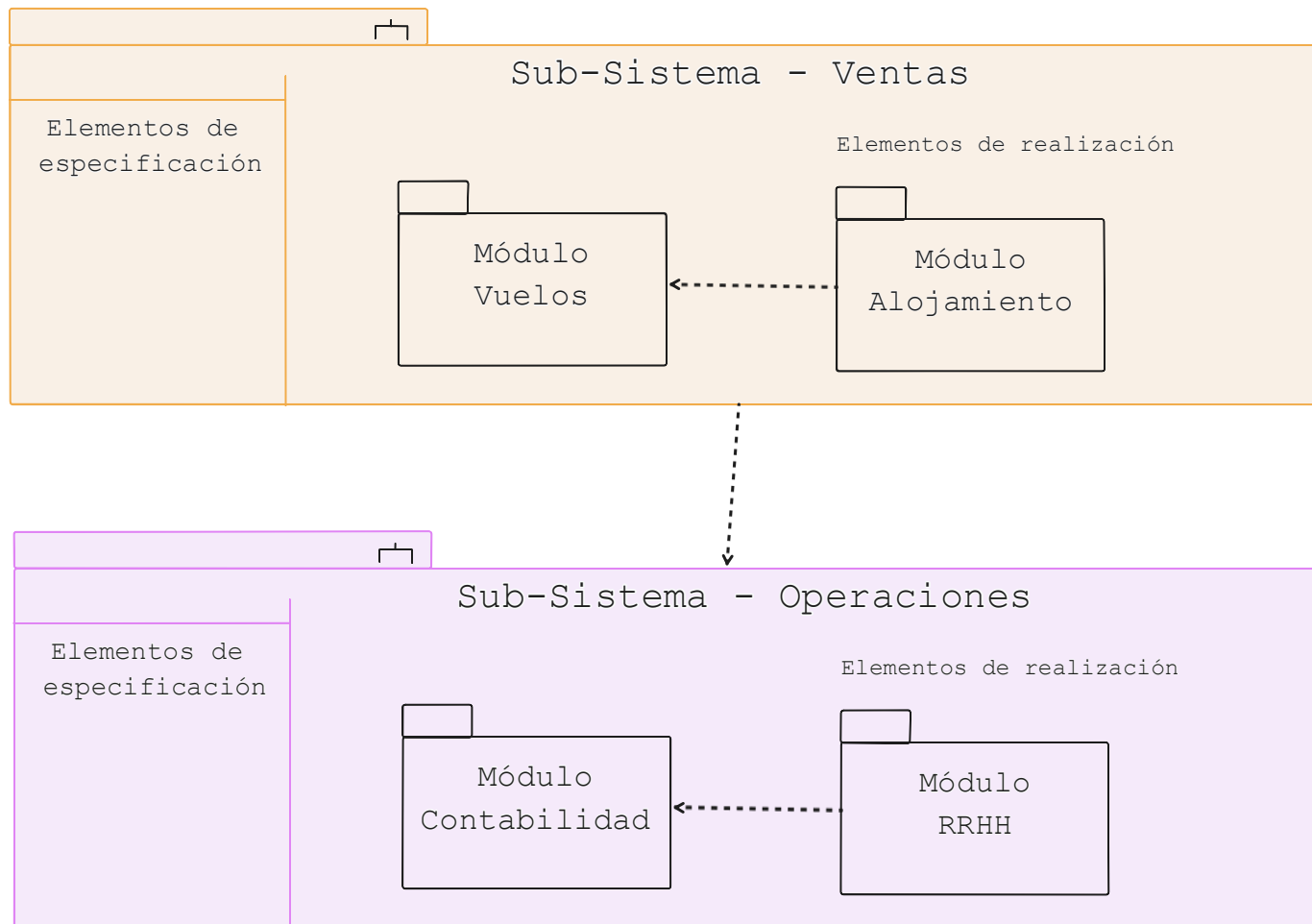


El proceso de diseño del software

- **Diseño arquitectónico:** Se define la arquitectura (generalmente se utiliza un patrón de arquitectura capas, MVC, monolítico, microservicios, SOA, etc).
- **Especificación abstracta:** Se especifican los subsistemas. Cada **subsistema** realiza **un servicio importante**
 - Contiene objetos altamente acoplados
 - Relativamente independiente de otros subsistemas
 - Generalmente se descompone en módulos aunque se puede descomponer en subsistemas más pequeños
- **Diseño de la interfaz:** Se describen las interfaces de los subsistemas. Permite que este se utilice como caja negra.
- **Diseño de los componentes:** Se descompone cada subsistema en componentes y se diseñan sus interfaces.

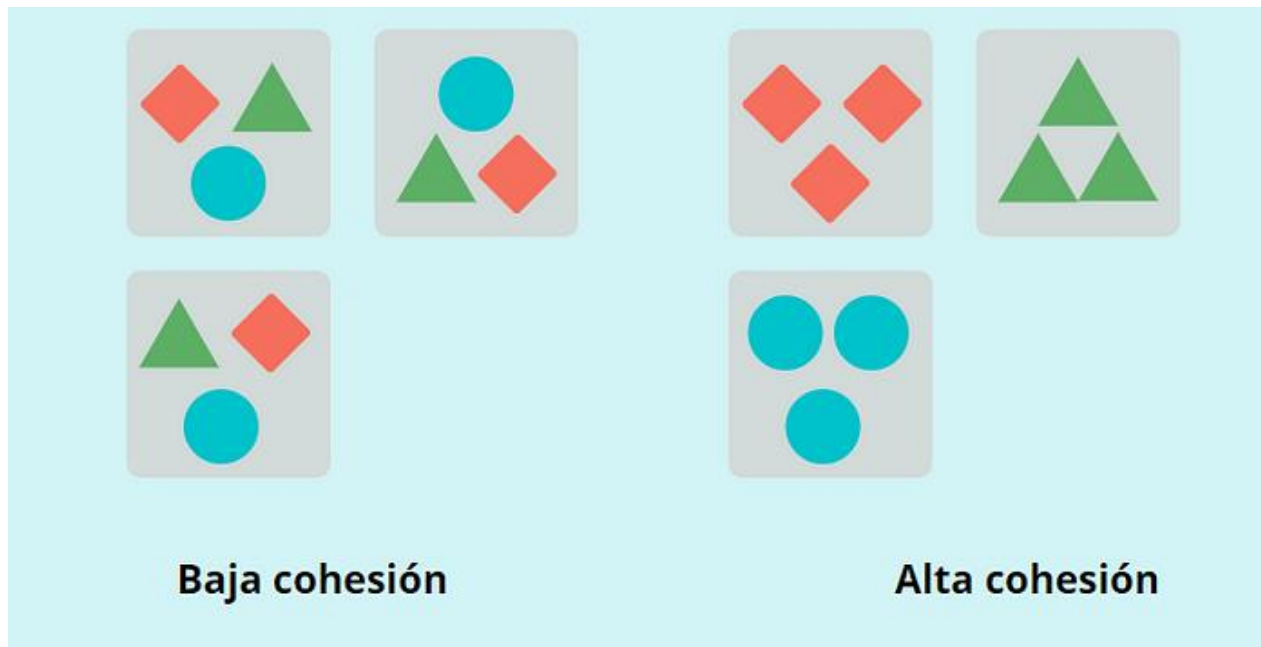
El proceso de diseño del Software

- Ejemplo Agencia de viajes
- Definimos los componentes dentro de los subsistemas



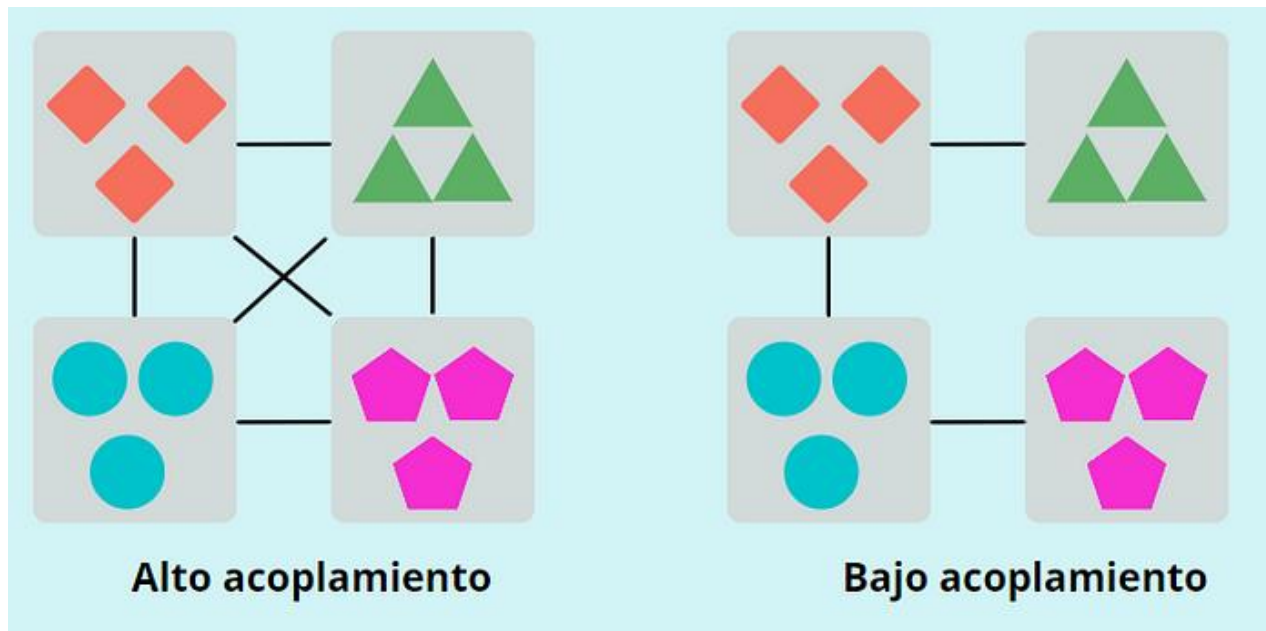
Principios del Diseño

- **Dividir y Conquistar:** Tratar con algo grande de entrada es normalmente mucho más difícil que si se hace con cosas más pequeñas
- **Incrementar la Cohesión:** Todo lo que sea posible. Un subsistema o módulo tiene un alto grado de cohesión si mantiene juntas las cosas que están relacionadas y afuera las restantes.



Principios del Diseño

- **Reducir el Acoplamiento:** Todo lo que sea posible. El acoplamiento ocurre cuando existe interdependencia entre un módulo y otro



- Mantener el **nivel de abstracción tan alto** como sea posible. Asegúrese que el diseño **oculte** o difiera consideraciones de detalle, reduciendo por tanto la complejidad

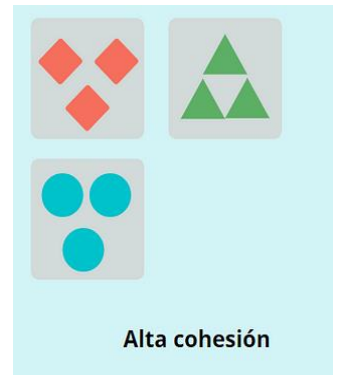
Principios del Diseño



- **Incrementar la reusabilidad** donde sea posible. Diseñe los variados aspectos de su sistema de tal forma que pueda ser usado nuevamente en otros contextos. Reutilización de los diseños y códigos existentes cuando sea posible.
- **Diseñe para ser flexible.** Activamente anticipe los cambios que el diseño pueda tener en el futuro, y prepararse para ello
- **Anticipe la obsolescencia.** Planifique los cambios en la tecnología o el entorno de tal manera que el software continúe ejecutándose o sea fácilmente cambiado. Por ej. Diseñar pantallas responsive.

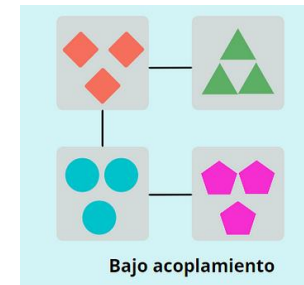
Cohesión

- Un subsistema o módulo tiene un alto grado de cohesión si mantiene “unidas” cosas que están relacionadas entre ellas y mantiene fuera el resto,
 - Un módulo cohesivo lleva a cabo una sola tarea dentro un procedimiento de software
- **Objetivo**
 - Diseñar servicios robustos y altamente cohesionados cuyos elementos estén fuerte y genuinamente relacionados entre sí
- **Ventajas**
 - Favorece la comprensión y el cambio de los sistemas
- El objetivo es entonces **maximizar la cohesión**



Acoplamiento

- Acoplamiento es la medida de la fortaleza de la asociación establecida por una conexión entre módulos dentro de una estructura de software
 - Depende de la complejidad de interconexión entre los módulos, el punto donde se realiza una entrada o referencia a un módulo y los datos que se pasan a través del interfaz.
 - Es una medida de la interconexión entre módulos dentro de una estructura del software
 - Un bajo acoplamiento indica un sistema bien dividido y puede conseguirse mediante la eliminación o reducción de relaciones innecesarias.



- Se produce una situación de acoplamiento cuando un elemento de un diseño depende de alguna forma de otro elemento del diseño.
 - **El objetivo es conseguir un acoplamiento lo más bajo posible**
 - Conseguir que cada componente sea tan independiente como sea posible

- Dos partes son co-nascentes si un cambio en una de ellas requiere modificar la otra para mantener la corrección general del sistema
 - Refina los conceptos de acoplamiento eferente y aferente
 - Estática: nombres de variables, clases, etc.; tipos ; significado o convención; posición; algoritmos.
 - Dinámica: orden de ejecución, tiempo, valores, identidad.

Clases de Acoplamiento

- Fuertemente acoplados
 - Si los módulos utilizan variables compartidas o intercambian información de control. Ver [co-nascencia](#).
- Débilmente acoplados. Ver [encapsulamiento](#).
 - Garantizar que los detalles de la representación de datos están dentro de un componente.
 - Interfaz con otros componentes mediante lista de parámetros.
 - Información compartida limitada a aquellos componentes que necesitan acceder a la información
 - Evitar compartir información de forma global.

Tarea de Aplicación 2

Acoplamiento y cohesión



Agenda — Parte 2

- Contexto... Qué es UML?
- Diagrama de casos de uso
- Diagramas de clase
- Diagramas de secuencia
- Diagramas de actividad
- Diagramas de
paquetes/componentes/despliegue

- El **Lenguaje Unificado de Modelado (UML)** es una familia de notaciones gráficas respaldada por un único metamodelo que ayuda a describir y diseñar sistemas de software, en particular aquellos contruidos utilizando el estilo orientado a objetos (OO).
- El UML es un término que puede significar cosas diferentes para diferentes personas, dependiendo de su experiencia y percepción de lo que hace que un proceso de ingeniería de software sea efectivo.
- A pesar de que los lenguajes de modelado gráfico han existido en la industria del software por mucho tiempo, hay mucho debate sobre su papel y el papel del UML en particular.
- El UML es un estándar relativamente abierto controlado por el Object Management Group (OMG), un consorcio abierto de empresas que se formó para construir estándares que apoyaran la interoperabilidad de los sistemas orientados a objetos. Desde su aparición en 1997, el UML ha unificado muchos lenguajes de modelado gráfico orientados a objetos y ha sido de gran ayuda para los desarrolladores de software.

Tipos de diagramas

Muestra las clases en un sistema, atributos y operaciones de cada clase y la relación entre cada clase.

**DIAGRAMA
DE
CLASE**

Muestra la relación estructural de los componentes de un sistema de software, que se comunican mediante interfaces.

**DIAGRAMA
DE
COMPONENTE**

Se utilizan para mostrar la estructura interna de una clase.

**DIAGRAMA
DE ESTRUCTURA
COMPUESTA**

Muestra el hardware de su sistema y el software de ese hardware, cada uno con su configuración de despliegue única.

**DIAGRAMA
DE
DESPLIEGUE**

Muestran la relación entre los objetos. Se utilizan para mostrar cómo se verá un sistema en un momento dado.

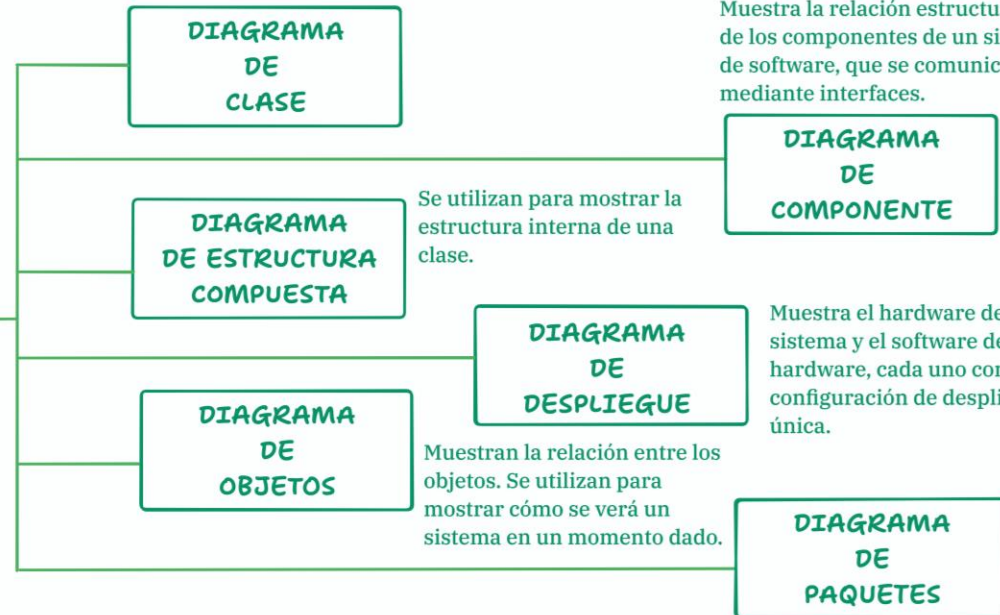
**DIAGRAMA
DE
OBJETOS**

**DIAGRAMA
DE
PAQUETES**

Muestra cómo un sistema está dividido en agrupaciones lógicas y las dependencias entre esas agrupaciones.

**DIAGRAMA
DE
ESTRUCTURA**

Los diagramas estructurales muestran la estructura estática del sistema y sus partes en diferentes niveles de abstracción.



Tipos de diagramas

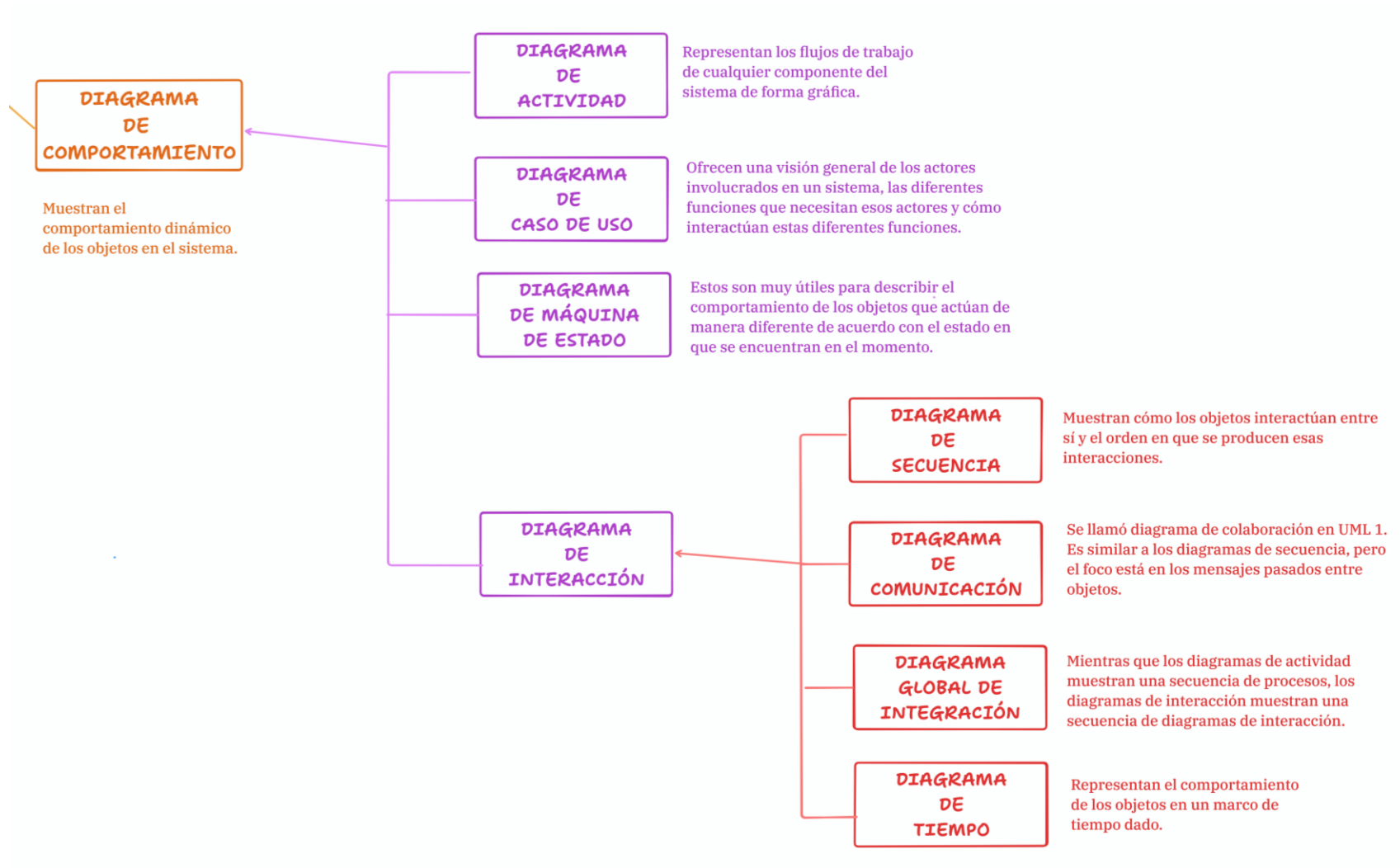


Diagrama de casos de uso

- Los casos de uso proporcionan una narrativa de cómo los actores utilizan el sistema.
 - **Representar los requisitos funcionales.**
 - Representar los **actores** que se comunican con el sistema. Normalmente los actores del sistema son los **usuarios** y **otros sistemas externos** que se relacionan con el sistema. En el caso de los usuarios hay que entender el actor como un “perfil”, pudiendo existir varios usuarios que actúan como el mismo actor.
 - **Representar las relaciones** entre requisitos funcionales y actores.

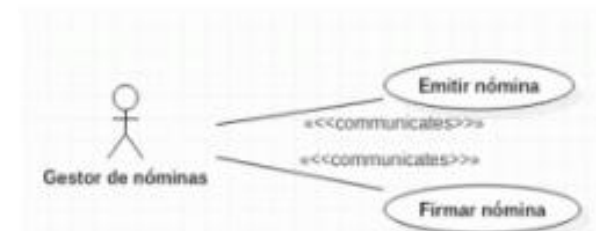


Diagrama de clase

- Los diagramas de clases UML son ampliamente utilizados y describen los **tipos de objetos** en un sistema, así como las **relaciones** estáticas entre ellos.
- Incluyen **propiedades, operaciones y restricciones** aplicables a las conexiones entre objetos.
- Los diagramas de clase **pueden tender a volverse incoherentes** a medida que se expanden y crecen. Es mejor evitar la creación de diagramas grandes y **dividirlos** en otros más pequeños que se puedan vincular entre sí más adelante.
- Usando la notación de clase simple, puedes crear rápidamente **una visión general de alto nivel** de su sistema.
- Cuantas más líneas se superpongan en sus diagramas de clase, más abarrotado se vuelve...
- Usa **colores** para agrupar módulos comunes. Diferentes colores en diferentes clases ayudan al lector a diferenciar entre los diversos grupos.

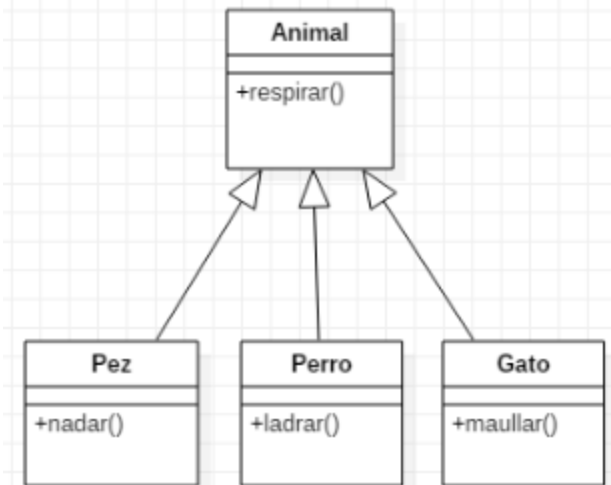


Diagrama de secuencia

- Su objetivo es representar **el intercambio de mensajes entre distintas instancias** del sistema para cumplir con una funcionalidad. Define, por tanto, el comportamiento dinámico del sistema de información.
- También se suele construir para comprender mejor el diagrama de clases, ya que el diagrama de secuencia muestra como objetos de esas clases interactúan haciendo intercambio de mensajes.

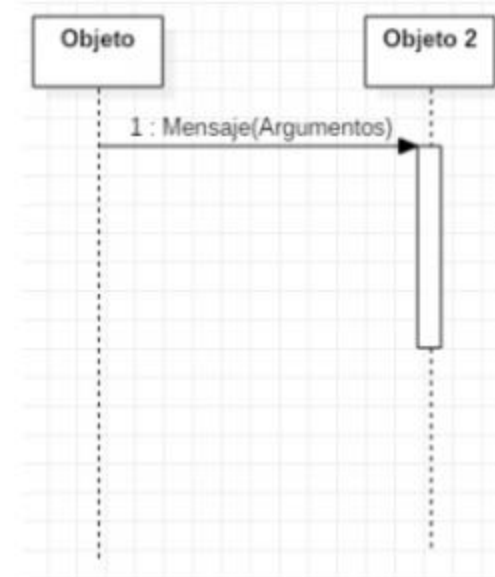


Diagrama de actividad



- Los diagramas de actividades muestran una secuencia de acciones, un **flujo de trabajo que va desde un punto inicial hasta un punto final**.
- La finalidad de este diagrama es modelar el **workflow** de una actividad a otra, pero sin tener en cuenta el paso de mensajes entre ellas.
- También es utilizado para modelar las actividades, que podemos asemejar a requisitos funcionales de negocio, por lo que este diagrama tendrá una influencia mayor a la hora de comprender el negocio o sus funcionalidades que en la propia implementación. Hay que tener en cuenta que este diagrama ofrece una visión a alto nivel.
 - Documentar los **requisitos** de negocio.
 - Modelar el **flujo de trabajo** entre actividades o/y entre subsistemas.
 - Comprender a **alto nivel** las funcionalidades del sistema de información.

Diagrama de actividad

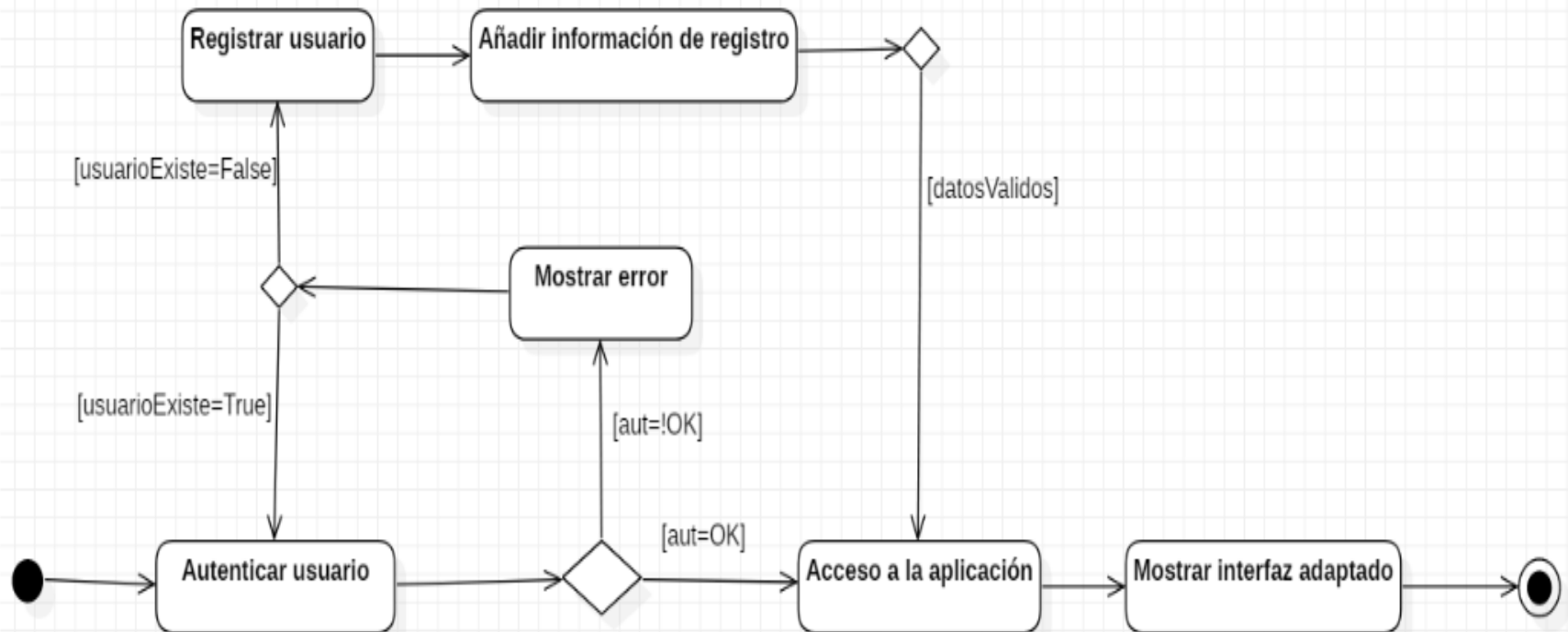


Diagrama de paquetes

- Los diagramas de clases sirven para representar la estructura más básica de un sistema.
- A medida que estos empiezan a crecer, necesitamos una forma más práctica de visualizar las relaciones y dependencias.
- Los diagramas de paquetes nos permiten agrupar por algún concepto de con un mayor nivel de abstracción.

Diagrama de paquetes

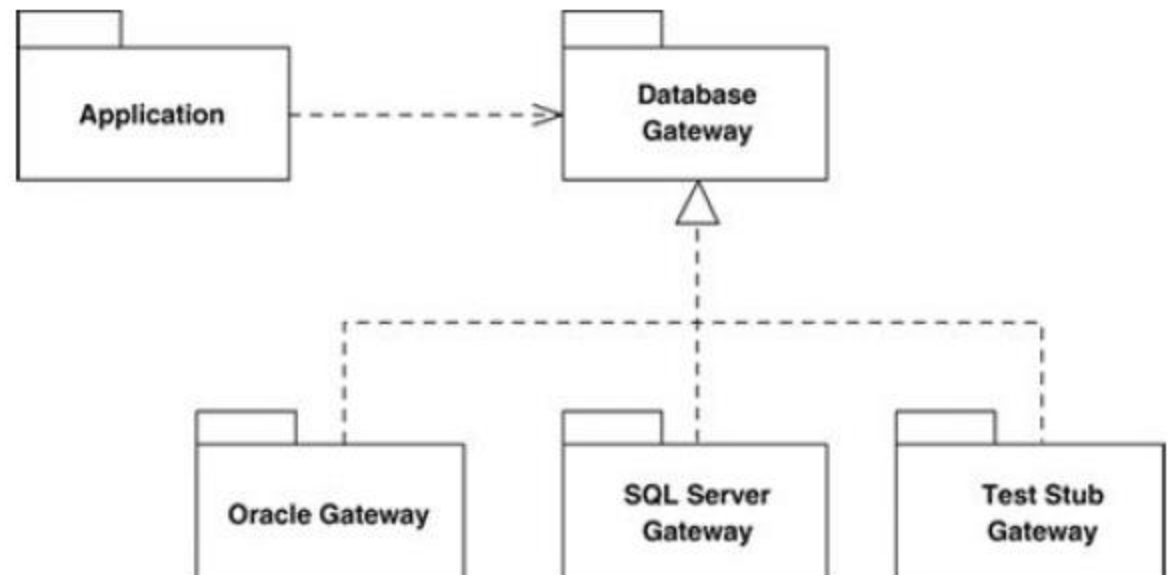
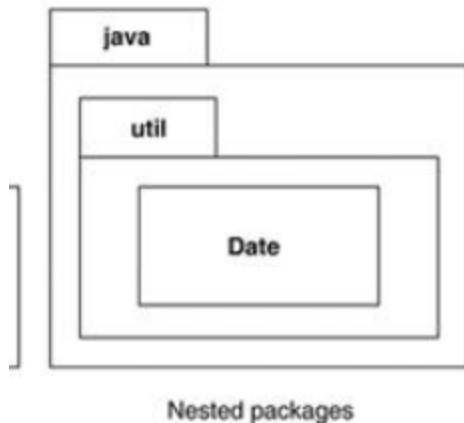


Diagrama de componentes

- Podemos verlo como una agrupación de módulos, por algún criterio.
- Sirve para documentar las relaciones entre el sistema a través de las interfaces.

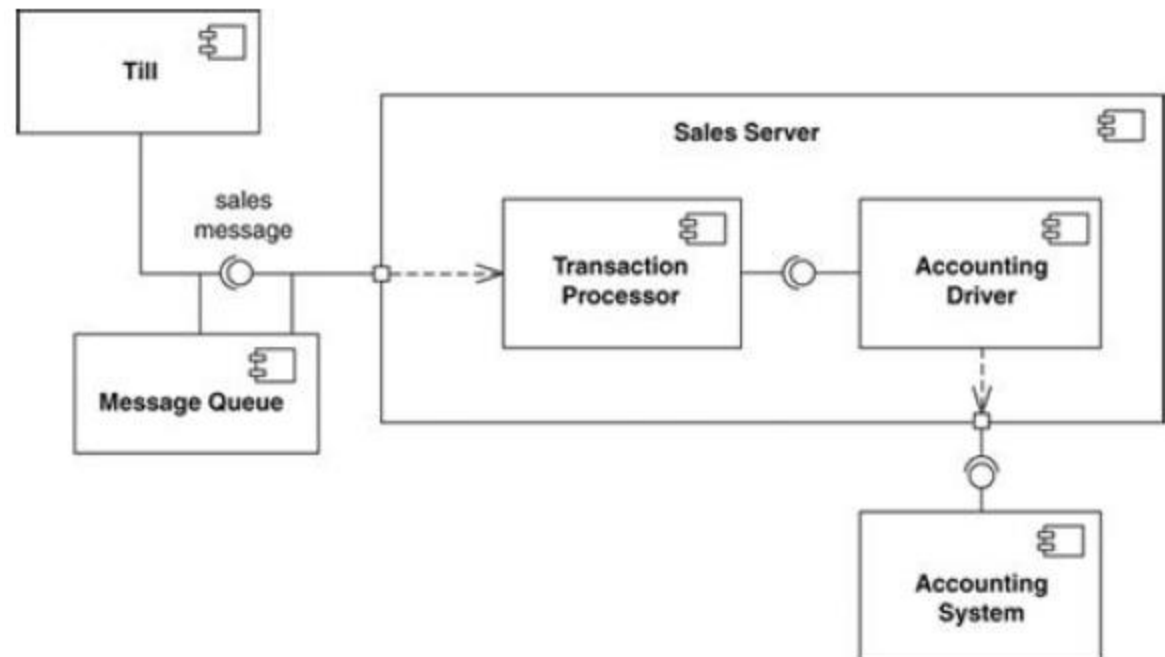
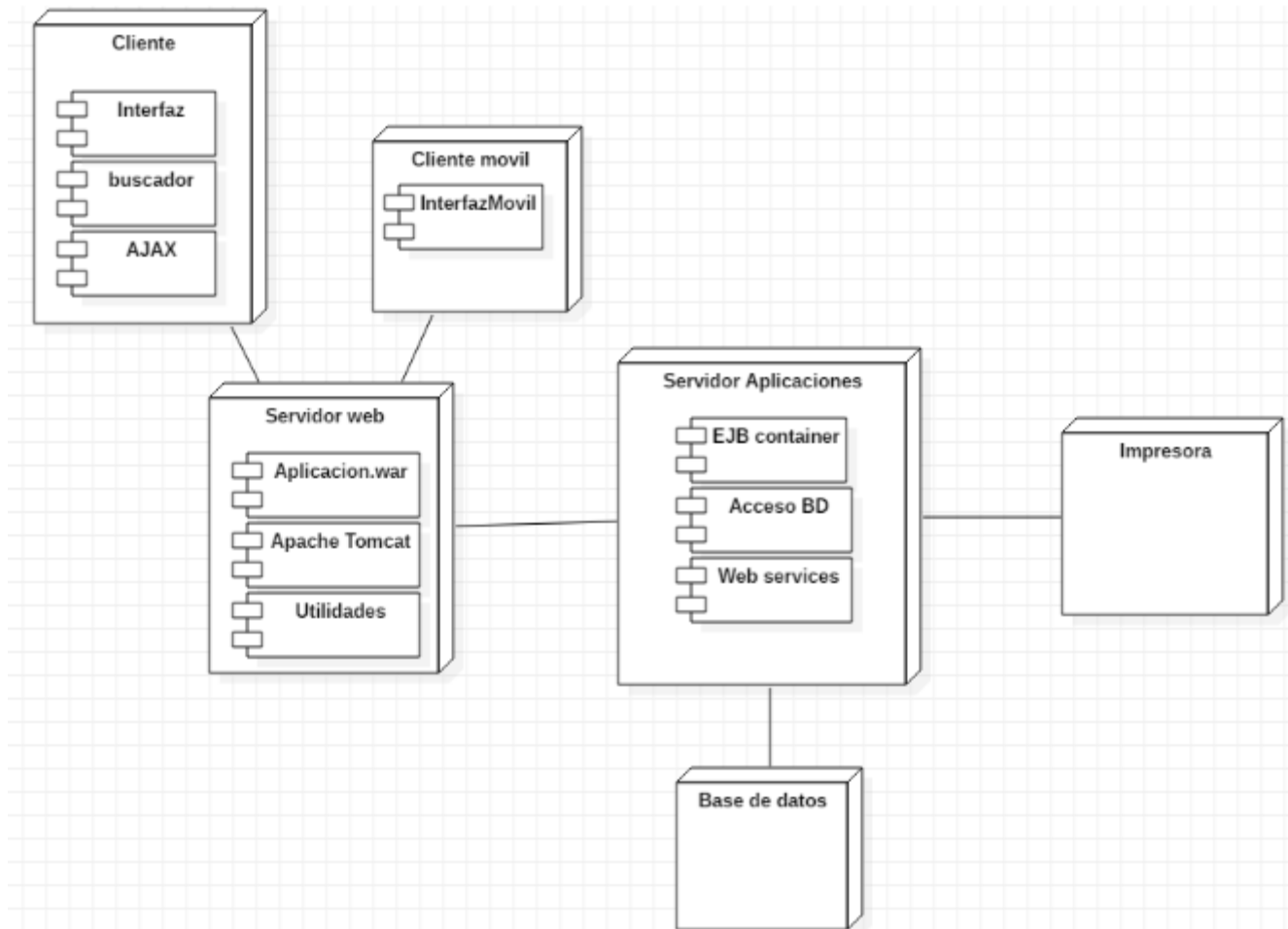


Diagrama de despliegue

- Suele ser utilizado junto con el diagrama de componentes (incluso a veces con el diagrama de paquetes) de forma que, juntos, dan una visión general de como estará desplegado el sistema de información.
- El diagrama de componentes muestra que componentes existen y como se relacionan mientras que el diagrama de despliegue es **utilizado para ver como se sitúan estos componentes lógicos en los distintos nodos físicos**.
- Como prácticamente todos los diagramas de UML, puede ser utilizado para representar aspectos generales o muy específicos, siendo utilizado de forma más común para aspectos generales.

Diagrama de despliegue



Tarea de Aplicación 3

Tipos de diagramas



- *Customer Journey map*
- Abstracción
- Encapsulamiento
- Modularidad
- Cohesión
- Acoplamiento
- Co-nascencia o *connascence*
- Herramientas de modelado y dibujo